

Lecture 08: Constructors

Initialisation, overloading, and the primary constructor

Agenda

1. What constructors are for
2. Constructor overloading (secondary constructors)
3. The primary constructor (idiomatic Kotlin)
4. `init` blocks and validation

Initialising objects

A **constructor** runs when you create an instance.

- Ensures the object starts in a valid state
- Receives construction arguments (name, id, capacity, ...)
- In Kotlin: *primary* constructor on the class header + optional *secondary* constructors in the body

Without a useful constructor

```
1 class User {  
2     var id: Int = 0  
3     var name: String = ""  
4 }  
5 fun main() {  
6     val u = User()  
7     u.id = 1  
8     u.name = "Ada"    // easy to forget a field  
9 }
```

Two-phase init is error-prone. Prefer constructing in one step.

Secondary constructor (first look)

```
1 class User {  
2     var id: Int  
3     var name: String  
4     constructor(id: Int, name: String) {  
5         this.id = id  
6         this.name = name  
7     }  
8 }  
9 fun main() {  
10     val u = User(1, "Ada")  
11 }
```

Quiz 1 — Role of constructors

Quiz

What is the main purpose of a constructor?

1. To destroy an object when the program ends
2. To initialise a new instance into a usable state
3. To convert Any to String
4. To replace the need for member functions

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. To initialise a new instance into a usable state.

Overloading

Multiple constructors with different parameter lists = **overload** set. The compiler picks by argument types/count. In Kotlin, secondary constructors must eventually delegate to the primary (if one exists) via `this(...)`.

Several secondary constructors

```
1 class Point {  
2     var x: Int  
3     var y: Int  
4     constructor() { x = 0; y = 0 }  
5     constructor(x: Int, y: Int) {  
6         this.x = x; this.y = y  
7     }  
8     constructor(v: Int) : this(v, v)  
9 }
```

Point(5) delegates to Point(5, 5).

When overloading helps

- Convenience: default origin, copy-from-pair, etc.
- Prefer **default parameters** on one constructor when variants only differ by optional values
- Too many overloads → hard to read; rethink the API

Quiz 2 — Which constructor runs?

Quiz

For the `Point` class on the previous slide, what are `x,y` after `Point(3)`?

1. 0, 0
2. 3, 0
3. 3, 3
4. Compile error

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

3. 3, 3

`constructor(v: Int) : this(v, v)` forwards to the two-argument constructor.

Idiomatic Kotlin: primary constructor

Declare construction parameters in the class header:

- `class User(val id: Int, var name: String)`
- `val/var` in the header declare **properties**
- Parameters without `val/var` are constructor-only (usable in `init` and property initialisers)

Primary constructor example

```
1 class User(val id: Int, var name: String) {  
2     init {  
3         require(name.isNotBlank()) { "name blank" }  
4     }  
5     fun display(): String = "#$id $name"  
6 }  
7  
8 fun main() {  
9     val u = User(7, "Grace")  
10    println(u.display())  
11 }
```

Header params without properties

```
1 class Temperature(celsius: Double) {  
2     val celsius: Double = celsius  
3     val fahrenheit: Double = celsius * 9 / 5 + 32  
4 }
```

celsius in the header is not a property unless marked `val`/`var`; here we assign to properties explicitly.

Secondary + primary together

```
1 class User(val id: Int, var name: String) {  
2     constructor(name: String) : this(0, name)  
3 }
```

Every secondary constructor must call `this(...)` (or another secondary that eventually does) so the primary always runs.

- Run as part of construction, interleaved with property initialisers
- Good place for validation (`require`, `check`)
- Multiple `init` blocks run in order top-to-bottom

Try this

Add `require(id >= 0)` to `User` and try constructing with `-1`.

Quiz 3 — Primary constructor

Quiz

In class `Book(title: String) { val title = title }`, is the header `title` a property?

1. Yes — all constructor parameters are properties
2. No — only `val`/`var` header params declare properties
3. Only if the class is `data class`
4. Only inside `init`

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. No — only `val`/`var` header params declare properties.

Here the property is the `val title` in the body; the header parameter merely feeds its initialiser.

Summary

- Constructors establish valid initial state
- Overload via secondary constructors; delegate with `this(...)`
- Prefer a **primary constructor** with `val`/`var` properties
- Use `init` + `require` for validation

Looking ahead

Next: inheritance, casting (up/down/smart), polymorphism, and a guided “accountant” practice using super.